

DC Lab — Experiment 6 Walkthrough (General Purpose)

Load Balancing Strategy: Least Connections

Goal: Implement a Load Balancer that sits in front of N identical backend servers and routes each incoming request to whichever backend currently has the **fewest active (in-flight) connections** — not a fixed rotation like Round Robin, but a decision made fresh for every request based on real-time load.

Why Least Connections matters: Round Robin assumes every request takes roughly the same amount of time to process. In reality, requests often have wildly different processing times (large vs small files, simple vs complex queries, etc.) — Round Robin can easily pile several slow requests onto one server while another sits idle. Least Connections adapts to this by always picking whichever backend is *currently* least busy.

Prerequisite: An activated Python `venv` with `grpcio` and `grpcio-tools` installed (same setup as any gRPC-based experiment).

Step 1 — Set up the project folder

```
mkdir dc_lab_exp6 && cd dc_lab_exp6
python3 -m venv venv
source venv/bin/activate
pip install grpcio grpcio-tools
mkdir proto
```

Step 2 — Define the service contract

```
proto/service.proto:
```

```
syntax = "proto3";
package lb_demo;
```

```
// A generic backend service. Any number of identical instances of
// this service can run simultaneously; the Load Balancer decides
// which instance handles each request.
```

```

service WorkerService {
    rpc HandleRequest (WorkRequest) returns (WorkReply);
}

message WorkRequest {
    int32 request_id = 1;
}

message WorkReply {
    int32 request_id = 1;
    string handled_by = 2;
    string status = 3;
}

```

Step 3 — Generate the Python stubs

```
python -m grpc_tools.protoc -I=proto --python_out=. --grpc_python_out=. proto/ser
```

Verify:

```
ls service_pb2*.py
```

Step 4 — Write the backend server (`worker_server.py`)

Each backend takes a port number and simulates variable-length work, so connection counts genuinely diverge over time:

```

import sys, time, random
from concurrent import futures
import grpc
import service_pb2, service_pb2_grpc

class WorkerServicer(service_pb2_grpc.WorkerServiceServicer):
    def __init__(self, name):
        self.name = name

    def HandleRequest(self, request, context):
        work_time = random.uniform(0.5, 2.5) # simulate variable processing tim
        print(f"[{self.name}] Handling request {request.request_id} "
              f"(will take {work_time:.1f}s)")
        time.sleep(work_time)
        print(f"[{self.name}] Finished request {request.request_id}")

```

```

        return service_pb2.WorkReply(
            request_id=request.request_id,
            handled_by=self.name,
            status="done",
        )

def serve(port):
    name = f"Worker-{port}"
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=20))
    service_pb2_grpc.add_WorkerServiceServicer_to_server(WorkerServicer(name), se
    server.add_insecure_port(f"localhost:{port}")
    server.start()
    print(f"{name} listening on localhost:{port}")
    try:
        while True:
            time.sleep(86400)
    except KeyboardInterrupt:
        server.stop(0)

if __name__ == "__main__":
    serve(int(sys.argv[1]))

```

Step 5 — Write the Load Balancer (`load_balancer.py`)

This is the actual algorithm — everything else in this experiment is plumbing around it:

```

import threading, time
import grpc
import service_pb2, service_pb2_grpc

BACKENDS = ["localhost:60201", "localhost:60202", "localhost:60203"]
NUM_REQUESTS = 9

active = [0] * len(BACKENDS)    # active[i] = current in-flight connections to BAC
lock = threading.Lock()

def pick_least_connections():
    """Core strategy: pick the backend with the fewest active
    connections right now, and increment its count immediately so
    the very next request sees the updated picture."""
    with lock:
        idx = active.index(min(active))

```

```

        active[idx] += 1
        print(f"[LB] Routing -> {BACKENDS[idx]} (active connections now: {active})
        return idx

def release(idx):
    with lock:
        active[idx] -= 1
        print(f"[LB] Released {BACKENDS[idx]} (active connections now: {active})"

def handle_request(req_id):
    idx = pick_least_connections()
    addr = BACKENDS[idx]
    try:
        with grpc.insecure_channel(addr) as channel:
            stub = service_pb2_grpc.WorkerServiceStub(channel)
            reply = stub.HandleRequest(service_pb2.WorkRequest(request_id=req_id)
            print(f"[LB] Request {req_id} -> handled by {reply.handled_by}")
    finally:
        release(idx)

def main():
    print(f"[LB] Load Balancer starting. Backends: {BACKENDS}")
    threads = []
    for req_id in range(1, NUM_REQUESTS + 1):
        t = threading.Thread(target=handle_request, args=(req_id,))
        threads.append(t)
        t.start()
        time.sleep(0.15)    # stagger arrivals, like real traffic

    for t in threads:
        t.join()

    print("\n[LB] All requests processed.")
    print(f"[LB] Final active connection counts (should all be 0): {active}")

if __name__ == "__main__":
    main()

```

The whole algorithm is `pick_least_connections()` — find the minimum, increment before dispatch, decrement on completion. Everything else is just running it and printing what happened.

Step 6 — Run it (needs 4 terminals)

Terminals 1–3 — start the backend servers, one per terminal, in any order, no timing pressure. Wait for each to print its “listening” line before starting the next:

```
# Terminal 1
cd dc_lab_exp6 && source venv/bin/activate
python worker_server.py 60201
```

```
# Terminal 2
cd dc_lab_exp6 && source venv/bin/activate
python worker_server.py 60202
```

```
# Terminal 3
cd dc_lab_exp6 && source venv/bin/activate
python worker_server.py 60203
```

Terminal 4 — once all three backends are listening, run the Load Balancer:

```
cd dc_lab_exp6 && source venv/bin/activate
python load_balancer.py
```

It dispatches 9 requests on its own (staggered every 0.15s) and finishes without needing further input.

Step 7 — What to check in the output

- Requests should **not** be evenly split in fixed request order (1→A, 2→B, 3→C, 4→A...) — instead, watch the `active connections` array printed with each routing decision. A backend gets picked because it currently has the lowest count, not because “it’s its turn”
- Over enough requests, load should still end up roughly balanced across all backends (e.g. 3/3/3 for 9 requests over 3 backends) — but the *reason* for that balance is real-time awareness, not a fixed pattern
- The final `active` array should return to `[0, 0, 0, ...]` — proving every connection was properly released after completion (no leaks in the counter)

Common mistakes

--	--	--

Mistake	Symptom	Fix
Starting the Load Balancer before all backends are up	<code>Connection refused</code> errors	Start Terminal 4 only after all 3 backends show "listening"
Incrementing the counter <i>after</i> dispatching instead of before	Two requests can both see the same "least loaded" backend and both get routed there	Increment <code>active[idx]</code> immediately inside the same locked block where you pick the minimum
Forgetting to release the connection count in a <code>finally</code> block	Counter drifts upward over time, routing decisions become wrong	Always decrement in a <code>finally</code> , so it happens even if the RPC call fails
No randomized work time in the backend	All backends look equally busy, Least Connections behaves just like Round Robin	Keep the <code>random.uniform(...)</code> simulated delay — that's what makes the strategy actually matter